



Python Programming Syllabus

From Zero to Confident — A Complete Beginner's Guide

Course Overview

This syllabus is designed for students with **no prior programming experience**. Topics progress gradually from foundational concepts to real-world Python applications.

What is Python?

Python is an interpreted, object-oriented programming language. Python is a high-level programming language. It is a dynamically typed language. It's high-level built-in data structures, combined with dynamic typing and dynamic binding. It is very useful for Rapid Application Development.

Features of Python Language

- **Python is beginner-friendly** — Simple, readable syntax that's easy to pick up for new programmers
- **Python is super flexible** — Supports multiple programming paradigms: procedural, object-oriented, and functional
- **Python is Easy to Learn** — Minimal boilerplate and clean code structure accelerate the learning curve
- **Python is highly versatile** — Used in web development, data science, automation, AI, scripting, and more
- **Free and Open-Source** — Freely available and backed by a large, active global community

Module 1: Introduction to Python

- What is Python? History and evolution
- Why Python? — Use cases and industries
- Python 2 vs Python 3 — key differences
- Installing Python and setting up the environment (VS Code / PyCharm)
- Writing and running your first Python program (`Hello, World!`)
- Python syntax, indentation rules, and code structure
- Comments — single-line (`#`) and multi-line (`"""`)
- Python interactive shell vs script mode

Module 2: Variables, Data Types & Operators

- What are variables? Naming rules and conventions
 - Core data types: `int`, `float`, `str`, `bool`, `NoneType`
 - Type checking with `type()` and type conversion (`int()`, `str()`, `float()`)
 - Taking user input with `input()`
 - Arithmetic operators: `+`, `-`, `*`, `/`, `//`, `%`, `**`
 - Comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=`
 - Logical operators: `and`, `or`, `not`
 - Assignment operators: `=`, `+=`, `-=`, etc.
 - String basics — indexing, slicing, concatenation, f-strings
-

Module 3: Control Flow — Conditions & Loops

- `if`, `elif`, `else` — conditional branching
 - Nested conditionals
 - `while` loop — syntax and use cases
 - `for` loop — iterating over sequences
 - `range()` function
 - Loop control: `break`, `continue`, `pass`
 - `else` clause in loops
 - Practical exercises: number guessing game, FizzBuzz
-

Module 4: Functions

- What is a function? Why use functions?
 - Defining and calling functions with `def`
 - Parameters vs arguments
 - Default parameter values
 - `return` statement and return values
 - Variable scope — local vs global
 - `*args` and `**kwargs`
 - Lambda (anonymous) functions
 - Recursion — concept and simple examples (factorial, Fibonacci)
 - Built-in functions: `print()`, `len()`, `range()`, `abs()`, `round()`, `sorted()`, etc.
-

Module 5: Strings In Depth

- String indexing and slicing
 - String methods: `.upper()`, `.lower()`, `.strip()`, `.split()`, `.join()`, `.replace()`, `.find()`, `.count()`
 - String formatting: f-strings, `.format()`, %-formatting
 - Multi-line strings
 - String immutability
 - Common string patterns and exercises
-

Module 6: Data Structures — Lists & Tuples

- **Lists**
 - Creating and accessing lists
 - List methods: `.append()`, `.extend()`, `.insert()`, `.remove()`, `.pop()`, `.sort()`, `.reverse()`
 - List slicing
 - List comprehensions
 - Nested lists
 - **Tuples**
 - Creating and accessing tuples
 - Tuple vs list — when to use which
 - Tuple unpacking
 - Named tuples (intro)
 - **Iterators and Generators**
 - `iter()` and `next()`
 - `yield` and generator functions
 - When and why to use generators
-

Module 7: Data Structures — Dictionaries & Sets

- **Dictionaries**
 - Creating dictionaries (key-value pairs)
 - Accessing, adding, updating, deleting entries
 - Dictionary methods: `.keys()`, `.values()`, `.items()`, `.get()`, `.update()`
 - Iterating over dictionaries
 - Nested dictionaries
 - Dictionary comprehensions

- **Sets**

- Creating sets
 - Set operations: union, intersection, difference, symmetric difference
 - Set methods: `.add()`, `.remove()`, `.discard()`
 - When to use sets (deduplication)
-

Module 8: Input, Output & File Handling

- Reading from files — `open()`, `read()`, `readline()`, `readlines()`
 - Writing to files — write mode (`w`) and append mode (`a`)
 - The `with` statement — context managers
 - Working with CSV files using the `csv` module
 - Working with JSON files using the `json` module
 - Binary file handling basics
 - Serialization with `pickle`
-

Module 9: Object-Oriented Programming (OOP)

- What is OOP? Procedural vs object-oriented thinking
 - Classes and objects — `class` keyword
 - The `__init__()` constructor
 - Instance variables and class variables
 - `self` — understanding its role
 - Instance methods, class methods (`@classmethod`), static methods (`@staticmethod`)
 - Encapsulation — public, protected, private attributes
 - Inheritance — single and multiple
 - Method overriding and `super()`
 - Polymorphism
 - Dunder (magic) methods: `__str__`, `__repr__`, `__len__`, `__eq__`
 - Abstract classes and interfaces (`abc` module)
 - Destroying objects — `__del__`
-

Module 10: Exception Handling

- Types of errors — syntax errors, runtime errors, logical errors
- What is an exception?
- `try`, `except`, `else`, `finally` blocks

- Catching specific exceptions (`ValueError` , `TypeError` , `ZeroDivisionError` , etc.)
 - Raising exceptions with `raise`
 - Custom (user-defined) exceptions
 - Python's built-in exception hierarchy
-

Module 11: Modules, Packages & the Standard Library

- What are modules? Importing with `import` and `from ... import`
 - Creating your own modules
 - Python packages — `__init__.py`
 - Useful standard library modules:
 - `os` — file system operations
 - `sys` — system-level operations
 - `math` — mathematical functions
 - `random` — random number generation
 - `datetime` — dates and times
 - `collections` — advanced data structures
 - `itertools` — efficient iteration
 - Installing third-party packages with `pip`
 - Virtual environments — `venv`
-

Module 12: Regular Expressions

- What are regular expressions?
 - The `re` module
 - Common patterns: `.` , `*` , `+` , `?` , `[]` , `^` , `$` , `{}`
 - `re.match()` vs `re.search()` vs `re.findall()`
 - Groups and capturing
 - `re.sub()` — search and replace
 - Practical use cases: email validation, phone number extraction
-

Module 13: Multithreading & Concurrency

- What is a process? What is a thread?
- The Global Interpreter Lock (GIL) — what it means
- Creating threads with the `threading` module
- Thread synchronization — Locks and Semaphores
- Deadlocks — what they are and how to avoid them

- Daemon threads
 - Introduction to `multiprocessing`
 - Intro to `asyncio` for async programming
-

Module 14: Working with Databases

- Introduction to databases and SQL basics
 - Connecting to MySQL using `mysql-connector-python`
 - CRUD operations — CREATE, READ, UPDATE, DELETE
 - Using `sqlite3` for lightweight local databases
 - DDL vs DML operations
 - Parameterized queries (preventing SQL injection)
 - Introduction to ORMs — `SQLAlchemy` basics
-

Module 15: Data Science Foundations with Python

- **NumPy**

- Arrays vs Python lists
- Creating and reshaping arrays
- Array indexing, slicing, and broadcasting
- Mathematical operations and universal functions
- Array I/O

- **Pandas**

- Series and DataFrames
- Reading CSV, Excel, and JSON files
- Data selection — `loc`, `iloc`
- Filtering, sorting, grouping data
- Handling missing values
- Merging and joining DataFrames
- Summary statistics

- **Matplotlib**

- Line charts, bar charts, histograms, scatter plots
 - Customizing plots — labels, titles, legends, colors
 - Saving figures
-

Module 16: Capstone Projects

Hands-on projects to consolidate all learning:

1. **Calculator App** — functions, control flow, user input
2. **To-Do List CLI App** — file I/O, lists, dictionaries
3. **Student Grade Manager** — OOP, file handling, exception handling
4. **Contact Book** — CRUD with SQLite, search with regex
5. **Basic Data Analysis Report** — Pandas + Matplotlib on a real dataset

***Tip for Students:** Consistency beats intensity. Write code every day, even if it's just 20 minutes. Build small projects early — they're the best teachers.*

Syllabus prepared for beginner Python learners — Python 3.x